



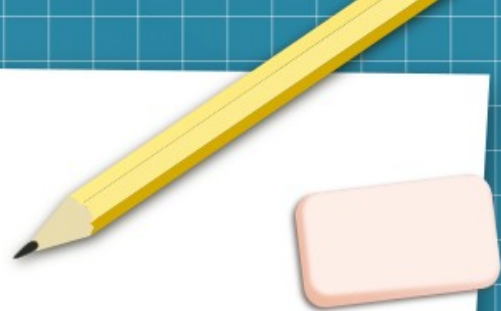
Lập trình di động Flutter

Bài 1

Khoa CNTT - Đại học Phenikaa

Nội dung trình bày

- Giới thiệu về Flutter
- Ngôn ngữ Dart
 - Các kiểu dữ liệu
 - Biến
 - Hằng
 - Toán tử
 - Các cấu trúc điều khiển
 - Kiểu List, Map
 - Hàm và Hàm Lambda



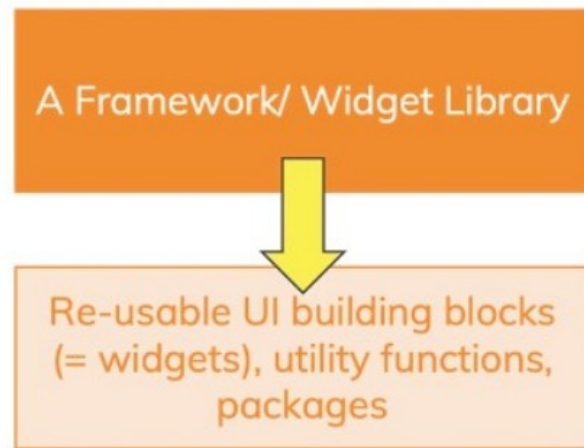
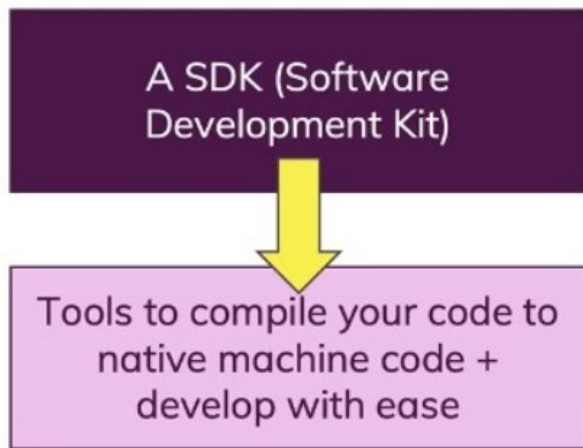
Flutter là gì ?

- **Flutter** được phát triển bởi Google và là công cụ cho phép xây dựng các ứng dụng mobile trên các nền tảng **công nghệ native và cross-platform** chỉ với **một ngôn ngữ lập trình**.
- Ứng dụng viết trên Flutter có thể chạy trên cả iOS và Android.



Flutter là gì ?

- Flutter đề cập đến 2 vấn đề



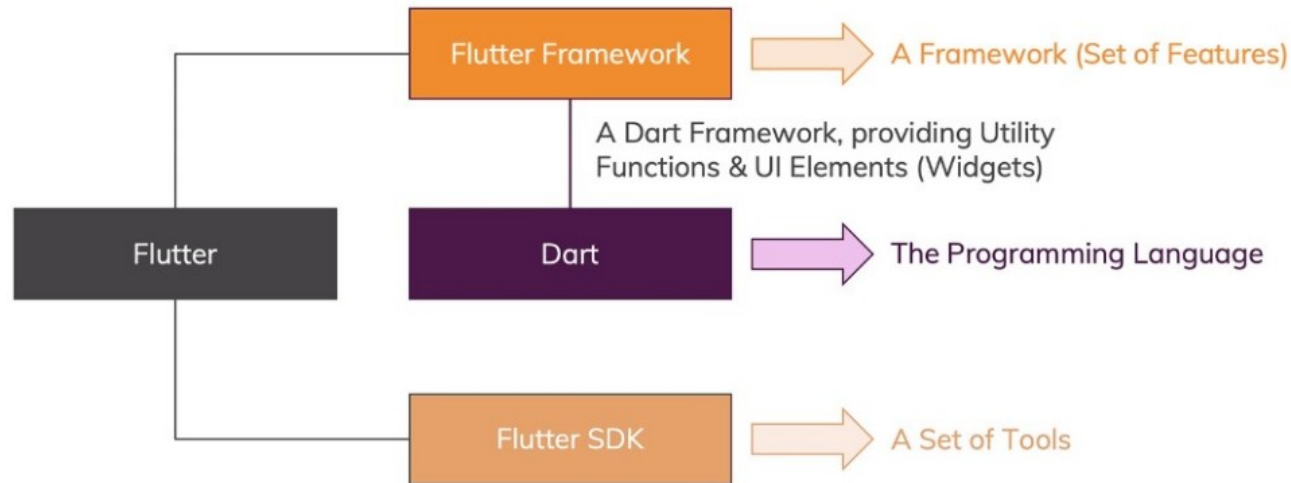
Dart là gì ?



- **Dart** là ngôn ngữ lập trình được phát triển bởi **Google**.
- **Flutter** sử dụng ngôn ngữ lập trình **Dart** để xây dựng giao diện người dùng **Front-end** (cho cả ứng dụng **Mobile và Web**).
- Ngôn ngữ Dart là độc lập với Flutter.
- Cú pháp của Dart là sự pha trộn giữa ngôn ngữ C và Java.

Flutter và Dart

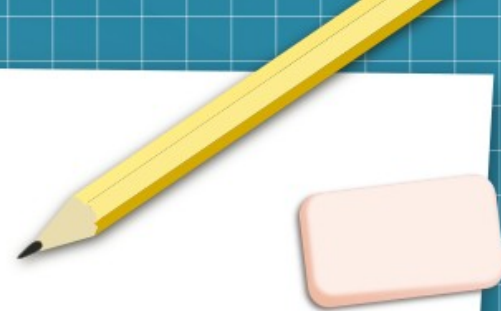
- Flutter chỉ là một tập hợp các công cụ, các tính năng, tiện ích được xây dựng bằng ngôn ngữ Dart. Do đó, ta có thể tận dụng lại các thành phần này để thay thế hay tùy chỉnh nó.



Ngôn ngữ Dart - Các kiểu dữ liệu

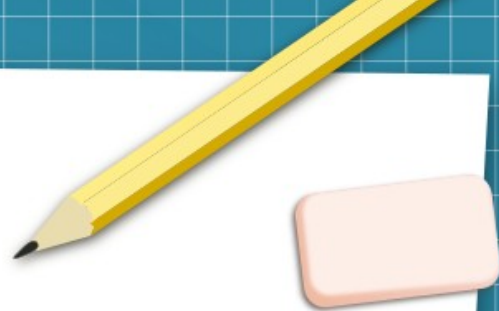
- Dart hỗ trợ các kiểu dữ liệu sau
 - Kiểu số
 - **int** : Số nguyên
 - **double** : Số thực 64 bits
 - Kiểu chuỗi ký tự
 - **String** : chuỗi ký tự của các ký tự dạng UTF-16
 - Kiểu logic
 - **bool** : chỉ nhận 2 giá trị true hoặc false

Biến



- Quy tắc đặt tên biến
 - Không trùng với từ khóa.
 - Có thể chứa chữ cái và chữ số.
 - Không chứa dấu cách và ký tự đặc biệt, ngoại trừ dấu gạch dưới (_) và ký hiệu đô la (\$).
 - Tên biến không được bắt đầu bằng số.
- Ví dụ:
 - Tên biến hợp lệ: **test\$123**, **test_123**
 - Tên biến không hợp lệ: **3test**, **3 test**, **for**

Biến



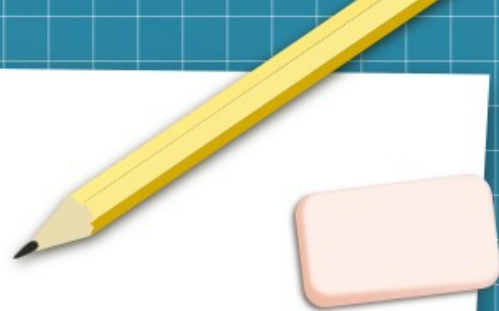
- Khai báo biến sử dụng kiểu dữ liệu có thể **null**.
 - Kiểu dữ liệu này không yêu cầu khởi tạo trước khi dùng và cho phép biến có thể **null**.
 - Cú pháp: **<kiểu dữ liệu>? <tên biến>**
 - Ví dụ:

```
int? value;  
print("$value");
```
- Khi chắc chắn rằng giá trị trả về không **null**, ta có thể thêm dấu chấm than (!) ở cuối để chuyển nó sang dạng không thể **null**.
 - Ví dụ:

```
int? nullable = 8;  
int notNullable = nullable!;
```
- Toán tử **??** có thể được dùng để tạo ra biến **không thể null** từ biến **có thể null**.
 - Ví dụ:

```
int? nullable = 10;  
int notNullable = nullable ?? 0; // Khi nullable bằng null thì notNullable bằng 0, ngược lại notNullable bằng nullable
```

Biến

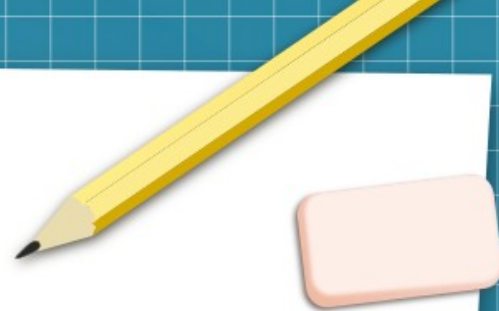


- Các biến trong Dart tham chiếu đến giá trị thay vì chứa giá trị.
- Sử dụng từ khóa **var** để khai báo một biến khi chưa xác định được kiểu giá trị cho nó.
 - Kiểu dữ liệu sẽ được xác định khi biến được nhận giá trị đầu tiên và sau đó kiểu dữ liệu sẽ không thay đổi được.
 - Ví dụ:

```
var name = "Tran Thu Lan";  
name = 1;    // Lỗi
```
- Từ khóa **dynamic** tương tự từ khóa **var** nhưng kiểu dữ liệu của biến được phép thay đổi trong quá trình thực hiện.
 - Ví dụ:

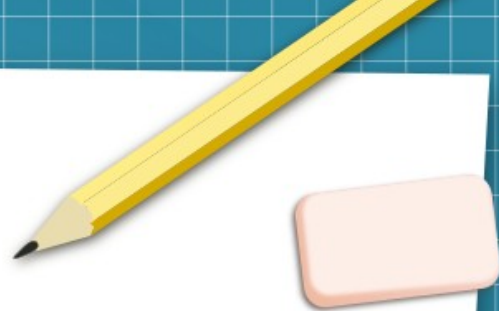
```
dynamic name = "Tran Thu Lan";  
name = 1;    // Không báo lỗi ở đây
```
- Các giá trị chưa được khởi gán ban đầu sẽ tự động nhận giá trị **null**.

Hằng



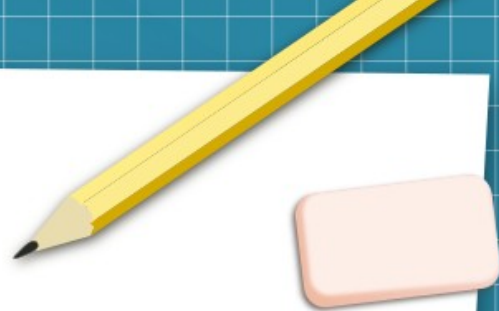
- Từ khóa **final** và **const** được sử dụng để khai báo các hằng số.
 - **final**: không cần cung cấp giá trị khởi tạo.
 - **const**: Bắt buộc phải cung cấp giá trị khởi tạo.
- Cú pháp:
final [<kiểu dữ liệu>] <tên hằng> [= <giá trị>];
const [<kiểu dữ liệu>] <tên hằng> = <giá trị>;
- Ví dụ:
final PI;
PI = 3.14;
final double PI = 3.14; // Xác định rõ kiểu dữ liệu cho PI
const E = 2.72;
const double E = 2.72;

Toán tử



- Các toán tử số học: +, -, *, /, ~/ , %, ++, --
 - ~/ : Chia lấy phần nguyên
- Các toán tử quan hệ: >, <, >=, <=, ==, !=
- Toán tử kiểm tra kiểu dữ liệu: is, is!
 - Ví dụ: **x is double** sẽ trả về **true** nếu **x** có kiểu dữ liệu là **double**
- Toán tử thực hiện bit: &, |, ^, ~, <<, >>
 - ^ : toán tử xor
 - ~ : toán tử not
 - << : dịch trái bit
 - >> : dịch phải bit
- Toán tử gán: =, ??=, +=, -=, *=, /=
 - ??= : chỉ gán được giá trị cho biến nếu biến bằng null
 - Ví dụ: **b ??= value; //** Lệnh này chỉ gán **value** cho **b** khi **b** là **null**.

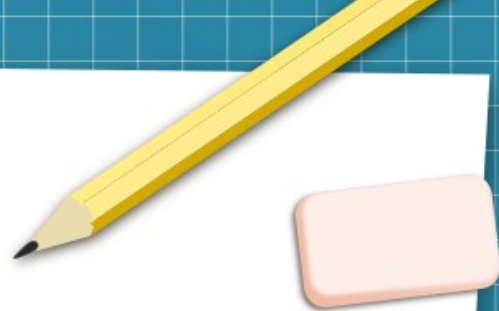
Toán tử



- Toán tử logic: &&, ||, !
- Toán tử điều kiện
 - **<điều kiện> ? <biểu thức 1> : <biểu thức 2>**
 - **<biểu thức 1> ?? <biểu thức 2>**
 - Nếu **<biểu thức 1>** khác **null** thì trả về giá trị của nó; ngược lại trả về giá trị của **<biểu thức 2>**
 - Ví dụ:

```
dynamic x;  
double y = x ?? 5.5; // y nhận giá trị 5.5  
x = 8.8;  
y = x ?? 5.5; // y nhận giá trị 8.8
```

Cấu trúc điều kiện và lặp



- Cấu trúc **if** và **switch** giống ngôn ngữ C
- Cấu trúc **for**, **while**, **do...while** giống ngôn ngữ C
- Cấu trúc **for ... in**

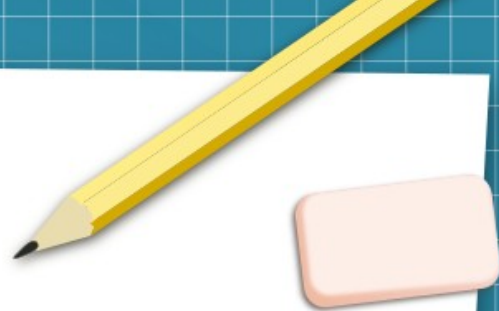
- Cú pháp:

```
for (<tên biến> in <đối tượng>) {  
    <lệnh thực hiện>  
}
```

- Ví dụ:

```
var obj = [12,13,14];  
for (var prop in obj) {  
    print(prop);  
}
```


Làm việc với kiểu số



- **int** : số nguyên
- **double** : kiểu số thực 64 bit theo chuẩn IEEE 754
- **num** : là kiểu kế thừa từ cả hai kiểu dữ liệu trên.
- Hàm **parse()** chuyển đổi một chuỗi thành một số.
 - Ví dụ: `num.parse("345");`

Làm việc với kiểu số

- Một số thuộc tính và hàm hỗ trợ

Sr.No	Property & Description
1	hashCode ↗ Returns a hash code for a numerical value.
2	isFinite ↗ True if the number is finite; otherwise, false.
3	isInfinite ↗ True if the number is positive infinity or negative infinity; otherwise, false.
4	isNan True if the number is the double Not-a-Number value; otherwise, false.
5	isNegative ↗ True if the number is negative; otherwise, false.
6	sign ↗ Returns minus one, zero or plus one depending on the sign and numerical value of the number.
7	isEven ↗ Returns true if the number is an even number.
8	isOdd ↗ Returns true if the number is an odd number.

Sr.No	Method & Description
1	abs ↗ Returns the absolute value of the number.
2	ceil ↗ Returns the least integer no smaller than the number.
3	compareTo ↗ Compares this to other number.
4	Floor ↗ Returns the greatest integer not greater than the current number.
5	remainder ↗ Returns the truncated remainder after dividing the two numbers.
6	Round ↗ Returns the integer closest to the current numbers.
7	toDouble ↗ Returns the double equivalent of the number.
8	toInt ↗ Returns the integer equivalent of the number.
9	toString ↗ Returns the string equivalent representation of the number.
10	truncate ↗ Returns an integer after discarding any fractional digits.

Làm việc với kiểu chuỗi ký tự

- Sử dụng String để khai báo một biến hay hằng có kiểu dữ liệu là chuỗi ký tự
 - Ví dụ: `String strName = "Michael Porter";`
- Sử dụng toán tử `+` để ghép 2 chuỗi
 - Ví dụ: `String testString = "Dai" + "hoc";`
- **`${<biểu thức>}`** đưa giá trị của biểu thức vào chuỗi
 - Ví dụ:
`int a = 8;`
`String str = "Gia tri cua a = $a";`
`String strBieuThuc = "Binh phuong cua $a = ${a * a}";`

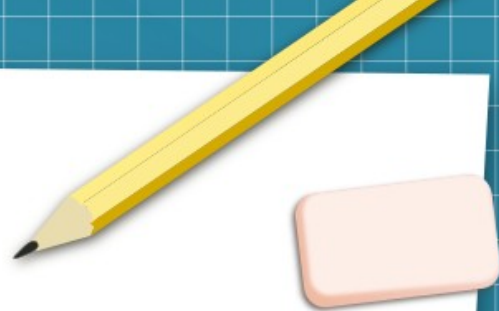
Làm việc với kiểu chuỗi ký tự

- Một số thuộc tính và hàm hỗ trợ

Sr.No	Property & Description
1	<code>codeUnits</code> ↗ Returns an unmodifiable list of the UTF-16 code units of this string.
2	<code>isEmpty</code> ↗ Returns true if this string is empty.
3	<code>Length</code> ↗ Returns the length of the string including space, tab and newline characters.

Sr.No	Methods & Description
1	<code>toLowerCase()</code> ↗ Converts all characters in this string to lower case.
2	<code>toUpperCase()</code> ↗ Converts all characters in this string to upper case.
3	<code>trim()</code> ↗ Returns the string without any leading and trailing whitespace.
4	<code>compareTo()</code> ↗ Compares this object to another.
5	<code>replaceAll()</code> ↗ Replaces all substrings that match the specified pattern with a given value.
6	<code>split()</code> ↗ Splits the string at matches of the specified delimiter and returns a list of substrings.
7	<code>substring()</code> ↗ Returns the substring of this string that extends from <code>startIndex</code> , inclusive, to <code>endIndex</code> , exclusive.
8	<code>toString()</code> ↗ Returns a string representation of this object.
9	<code>codeUnitAt()</code> ↗ Returns the 16-bit UTF-16 code unit at the given index.

Làm việc với kiểu List



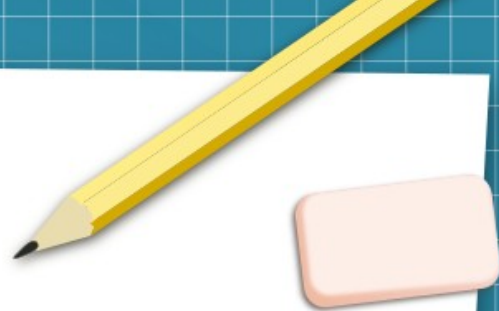
- Kiểu **List** trong Dart tương tự kiểu mảng ở các ngôn ngữ khác.
- Khai báo:
 - `var <tên List> = [<giá trị 1>, <giá trị 2>, ...];`
 - `List<kiểu dữ liệu> <tên List> = [<giá trị 1>, <giá trị 2>, ...];`
 - `var <tên list> = List.filled(size, <default value>);`
- Ví dụ:
 - `var myListNum = [5, 6, 9];`
 - `var myListNum = new List.filled(3, 0);`
- Truy cập List thông qua chỉ số bắt đầu từ 0
- Ví dụ: `myListNum[0]`, `myListNum[1]`, ...

Làm việc với kiểu List

- Các thuộc tính

Sr.No	Methods & Description
1	<code>first</code> ↗ Returns the first element in the list.
2	<code>isEmpty</code> ↗ Returns true if the collection has no elements.
3	<code>isNotEmpty</code> ↗ Returns true if the collection has at least one element.
4	<code>length</code> ↗ Returns the size of the list.
5	<code>last</code> ↗ Returns the last element in the list.
6	<code>reversed</code> ↗ Returns an iterable object containing the lists values in the reverse order.
7	<code>Single</code> ↗ Checks if the list has only one element and returns it.

Làm việc với kiểu List



- Các thao tác cơ bản trên List
 - Thêm: **add()**, **addAll()**, **insert()**, **insertAll()**
 - Cập nhật: **replaceRange()**
 - Xóa: **remove()**, **removeAt()**, **removeLast()**, **removeRange()**
- Tự tra cứu cách thực hiện của các hàm trên.
- In các phần tử của List lên màn hình

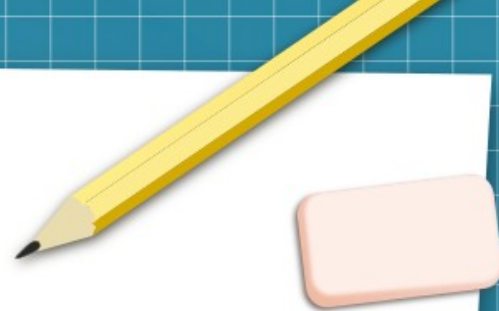
```
for(var item in myListNum)
    print(item);
```

Làm việc với kiểu Map



- Map đại diện cho một cấu trúc dữ liệu bao gồm các ánh xạ giữa các **khoá** (key) và các **giá trị** (value). Các **khoá** không được phép trùng nhau, và mỗi **khoá** sẽ tương ứng với một **giá trị**.
- Khai báo:
 - `var <tên Map> = {key1:value1, key2:value2, ...};`
 - `var <tên Map> = new Map();`
- Ví dụ:
 - `var info = {'MaSV': '20012002', 'Ten': 'Thanh'};`
 - `info['DiaChi'] = 'Ha Noi';`

Làm việc với kiểu Map



- Một số thuộc tính và hàm hỗ trợ

Sr.No	Property & Description
1	Keys ↗ Returns an iterable object representing keys
2	Values ↗ Returns an iterable object representing values
3	Length ↗ Returns the size of the Map
4	isEmpty ↗ Returns true if the Map is an empty Map
5	isEmpty ↗ Returns true if the Map is an empty Map

Sr.No	Function Name & Description
1	addAll() ↗ Adds all key-value pairs of other to this map.
2	clear() ↗ Removes all pairs from the map.
3	remove() ↗ Removes key and its associated value, if present, from the map.
4	forEach() ↗ Applies f to each key-value pair of the map.

Hàm

- **Hàm** trong Dart giống ngôn ngữ C, cách gọi hàm cũng như vậy.
- Ví dụ:

```
void newPrint() {  
    print("Function Called");  
}
```

```
num sum(num x, num y) {  
    return x+y;  
}
```

Tham số mặc định của hàm

- Xét ví dụ:

```
printer(num n, {String? s1, String? s2}) {  
    print(n);  
    print(s1);  
    print(s2);  
}
```

```
main() {  
    printer(75, s1: 'hello', s2: 'there');  
    printer(25, s1: 'hello');           // s2 sẽ nhận đầu ra là null  
}
```

Hàm Lamda

- **Hàm Lamda** hay còn gọi là **Hàm mũi tên** là một cơ chế biểu diễn hàm một cách ngắn gọn.
- Cú pháp:
 - <giá trị trả về> <tên hàm>(<tham số>) => <biểu thức>;
 - Ví dụ:

```
printMsg() => print("hello");  
int tong(var a, var b) => a + b;
```


Q & A

